

CompanionIO - Système d'Assistance Intelligent

Mahdousha

May 21, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction et Présentation du Projet

1.1 Contexte et Motivation

L'intelligence artificielle conversationnelle représente l'une des avancées technologiques les plus significatives de ces dernières années. Les assistants virtuels comme Siri, Alexa ou Google Assistant ont démontré le potentiel de l'IA pour faciliter les interactions homme-machine. Cependant, ces solutions restent souvent limitées à des cas d'usage spécifiques et ne constituent pas de véritables partenaires numériques capables d'orchestrer des tâches complexes de manière autonome.

Dans ce contexte, le projet **CompanionIO** s'inscrit comme une réponse innovante aux limitations des assistants traditionnels. Il propose un système d'assistance intelligent capable de comprendre, planifier et exécuter des tâches complexes tout en maintenant une interaction conversationnelle naturelle avec l'utilisateur.

1.2 Présentation Générale du Système

CompanionIO est une plateforme d'assistance intelligente conçue pour accompagner l'utilisateur dans ses activités quotidiennes numériques. Contrairement aux assistants traditionnels qui se limitent à des réponses ponctuelles, CompanionIO vise à offrir une expérience continue et intégrée.

Le système repose sur une architecture microservices moderne déployée sur Microsoft Azure, combinant :

- Intelligence artificielle conversationnelle avancée
- Traitement automatique du langage naturel (NLP)
- Reconnaissance et synthèse vocale
- Orchestration de tâches complexes
- Interfaces multi-modales (texte, voix, interfaces graphiques)

1.2.1 Objectifs Principaux

Le système poursuit plusieurs objectifs ambitieux :

1. **Interaction Naturelle** : Permettre une communication fluide et intuitive avec l'utilisateur
2. **Orchestration Intelligente** : Coordonner automatiquement des tâches complexes
3. **Apprentissage Continu** : S'adapter aux préférences et habitudes de l'utilisateur
4. **Sécurité et Confidentialité** : Garantir la protection des données personnelles
5. **Extensibilité** : Faciliter l'ajout de nouvelles fonctionnalités

1.2.2 Cas d'Usage Cibles

CompanionIO s'adresse à différents scénarios d'utilisation :

- Gestion d'agenda et planification de tâches
- Recherche et synthèse d'informations
- Automatisation de workflows professionnels
- Assistance dans les tâches quotidiennes
- Interface unifiée vers divers services numériques

1.3 État de l'Art et Positionnement

1.3.1 Limites des Solutions Existantes

L'analyse des assistants conversationnels existants révèle plusieurs limitations majeures :

1. **Spécialisation Excessive** : La plupart des assistants sont conçus pour des tâches spécifiques
2. **Interaction Réactive** : Nécessité d'une sollicitation explicite de l'utilisateur
3. **Confinement aux Écosystèmes** : Dépendance forte aux plateformes propriétaires
4. **Limitations en Orchestration** : Difficulté à coordonner des tâches complexes
5. **Problèmes de Confidentialité** : Collecte massive de données personnelles

1.3.2 Apports du Projet

CompanionIO se distingue par plusieurs innovations :

- **Architecture Distribuée** : Microservices scalables et résilients
- **Approche Stateless** : Séparation claire entre état et logique applicative
- **Sécurité Renforcée** : Authentification robuste et contrôle d'accès granulaire
- **Extensibilité Modulaire** : Architecture facilitant l'ajout de nouvelles capacités
- **Orchestration Avancée** : Coordination intelligente de tâches complexes

1.4 Méthodologie et Approche

1.4.1 Méthodes Agiles

Le développement du projet suit une approche agile inspirée de Scrum, permettant :

- Itérations courtes et livrables fréquents
- Adaptation continue aux retours utilisateurs
- Collaboration étroite entre équipes
- Focus sur la qualité et la maintenabilité

1.4.2 Technologies et Outils

Le système s'appuie sur un écosystème technologique moderne :

- **Langages** : Python (FastAPI), C# (.NET 8), JavaScript
- **Infrastructure** : Docker, Kubernetes, Azure Container Apps
- **Base de Données** : PostgreSQL, Redis
- **IA/ML** : Modèles de langage avancés (GPT, BERT)
- **Communication** : WebSockets, REST APIs, Message Queues

1.4.3 Contraintes et Défis

Le projet doit relever plusieurs défis techniques et organisationnels :

1. Gestion de la latence dans les interactions temps réel
2. Orchestration de services distribués hétérogènes
3. Sécurité et confidentialité des données
4. Scalabilité et disponibilité du système
5. Intégration de modèles d'IA en évolution constante

1.5 Structure de la Documentation

Cette documentation est organisée selon la structure suivante :

1. **Chapitre 2** : Architecture générale du système
2. **Chapitre 3** : Service Orchestrateur - cœur du système
3. **Chapitre 4** : Service d'Authentification et Autorisation
4. **Chapitre 5** : Service de Traitement du Langage (LLM)

5. **Chapitre 6** : Service de Reconnaissance Vocale (STT)
6. **Chapitre 7** : Service de Synthèse Vocale (TTS)
7. **Chapitre 8** : Environnement de développement
8. **Chapitre 9** : Pipeline CI/CD
9. **Chapitre 10** : Tests et Assurance Qualité
10. **Chapitre 11** : Sécurité et Authentification
11. **Chapitre 12** : Déploiement et Production
12. **Chapitre 13** : Monitoring et Observabilité
13. **Chapitre 14** : Conclusion et Perspectives

Chaque chapitre inclut des espaces réservés pour les captures d'écran qui seront ajoutées ultérieurement.

1.6 Conclusion

CompanionIO représente une avancée significative dans le domaine des assistants intelligents, en proposant une alternative aux solutions existantes grâce à une architecture moderne et une approche centrée sur l'utilisateur. Le système vise à devenir un véritable partenaire numérique, capable d'anticiper les besoins et d'orchestrer des tâches complexes de manière autonome.

Le chapitre suivant présente l'architecture générale du système et les choix technologiques qui sous-tendent sa conception.

Chapter 2

Architecture Générale du Système

2.1 Principes Architecturaux

CompanionIO repose sur une architecture microservices moderne conçue pour assurer scalabilité, résilience et maintenabilité. Cette section présente les principes fondamentaux qui guident la conception du système.

2.1.1 Architecture Microservices

L'approche microservices permet de décomposer le système en composants indépendants, chacun responsable d'une fonctionnalité spécifique :

- **Orchestrateur** : Coordination et routage des requêtes
- **Authentification** : Gestion des utilisateurs et autorisations
- **LLM** : Traitement du langage naturel
- **STT** : Reconnaissance vocale
- **TTS** : Synthèse vocale

2.1.2 Approche Stateless

Le système maintient une séparation stricte entre logique applicative et état :

- **État Externalisé** : Stockage de l'état dans Redis/PostgreSQL
- **Services Stateless** : Possibilité de redémarrage sans perte de données
- **Scalabilité Horizontale** : Ajout de instances sans reconfiguration

2.1.3 Communication Asynchrone

Les services communiquent via des mécanismes asynchrones :

- **WebSockets** : Communication temps réel
- **Message Queues** : Découplage des services
- **APIs REST** : Interfaces synchrones pour les opérations critiques

2.2 Architecture Globale

2.2.1 Diagramme d'Architecture

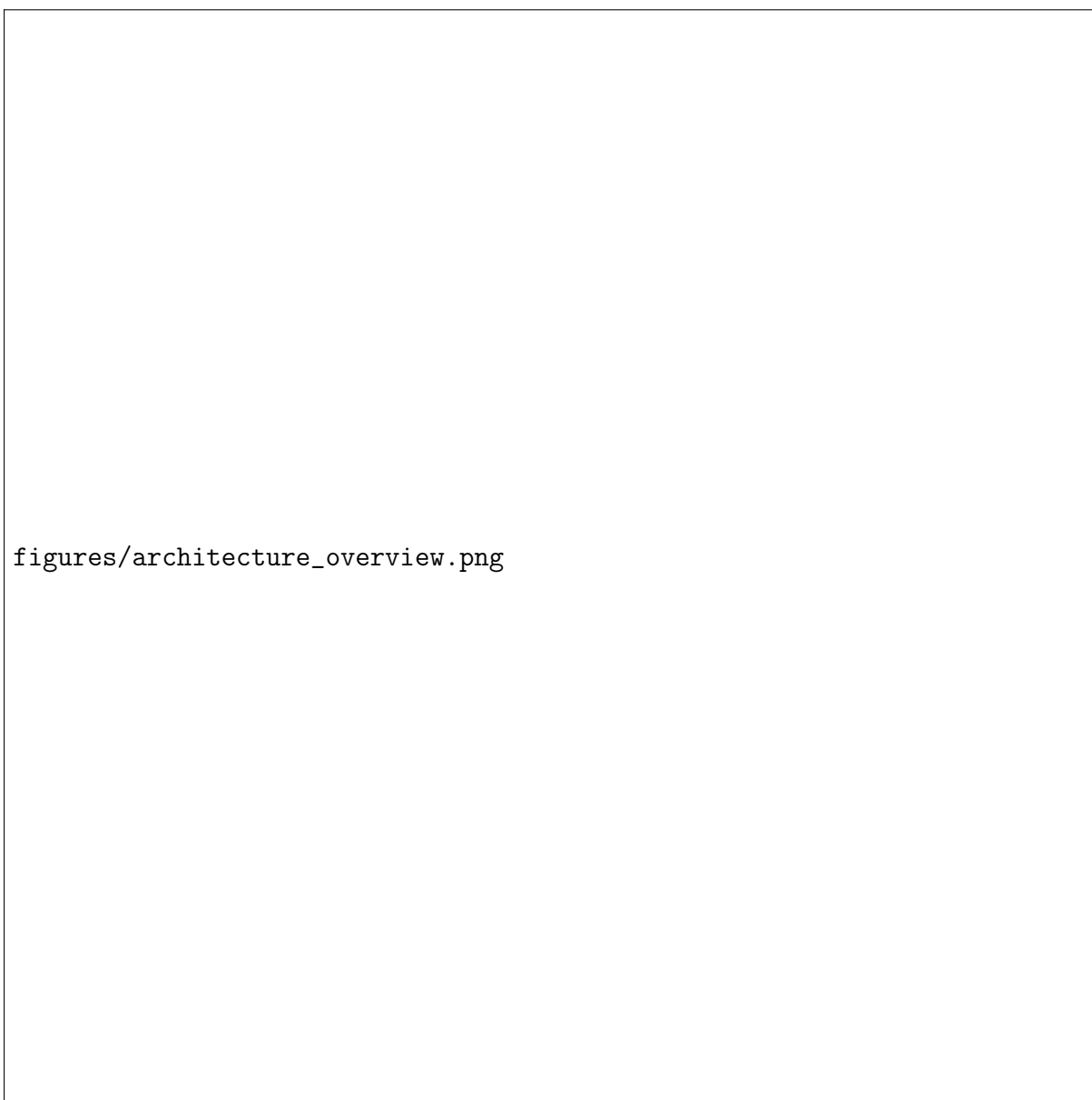


Figure 2.1: Architecture générale de CompanionIO

Le système est organisé autour de plusieurs couches :

1. **Couche Interface** : Points d'entrée utilisateur (Web, Mobile, API)
2. **Couche Orchestration** : Coordination et routage intelligent
3. **Couche Services** : Fonctionnalités métier (LLM, STT, TTS, Auth)
4. **Couche Infrastructure** : Bases de données, cache, stockage

2.2.2 Flux de Données

Le flux de données dans le système suit un pattern complexe mais optimisé :

1. L'utilisateur initie une requête via une interface
2. L'orchestrateur valide l'authentification et route la requête
3. Les services appropriés traitent la demande
4. Les résultats sont agrégés et retournés à l'utilisateur
5. L'état est persisté de manière asynchrone

2.3 Composants Techniques

2.3.1 Infrastructure Cloud

Le système est déployé sur Microsoft Azure avec les services suivants :

- **Azure Container Apps** : Hébergement des microservices
- **Azure Database for PostgreSQL** : Base de données relationnelle
- **Azure Cache for Redis** : Cache distribué et files d'attente
- **Azure Container Registry** : Registre d'images Docker
- **Azure Front Door** : Routage et équilibrage de charge

2.3.2 Technologies de Base

Langages et Frameworks

Service	Langage	Framework
Orchestrateur	Python 3.14	FastAPI
Authentification	C#	ASP.NET Core 8
LLM	Python 3.14	FastAPI
STT	Python 3.14	FastAPI
TTS	Python 3.14	FastAPI

Table 2.1: Technologies utilisées par service

Base de Données et Cache

- **PostgreSQL** : Données relationnelles (utilisateurs, sessions, métadonnées)
- **Redis** : Cache, sessions, files d'attente, données temporaires
- **Stockage Objet** : Azure Blob Storage pour fichiers multimédias

2.3.3 Communication Inter-Services

Protocoles Utilisés

Type	Protocole	Usecase
Synchrone	HTTP/REST	Opérations critiques
Asynchrone	WebSockets	Communication temps réel
Messages	Redis Streams	Événements et notifications

Table 2.2: Protocoles de communication

2.4 Sécurité et Authentification

2.4.1 Modèle de Sécurité

Le système implémente une sécurité multi-couches :

- **Authentification** : JWT tokens avec OAuth 2.0 (Google)
- **Autorisation** : Role-Based Access Control (RBAC)
- **Chiffrement** : TLS 1.3 pour toutes les communications
- **Audit** : Traçabilité complète des actions

2.4.2 Gestion des Secrets

- **Azure Key Vault** : Stockage sécurisé des secrets
- **Injection Runtime** : Secrets injectés au démarrage
- **Rotation Automatique** : Mise à jour périodique des clés

2.5 Scalabilité et Performance

2.5.1 Stratégies de Scalabilité

1. **Scalabilité Horizontale** : Ajout d'instances selon la charge
2. **Cache Distribué** : Redis pour réduire la latence
3. **Load Balancing** : Distribution intelligente des requêtes
4. **Auto-scaling** : Adaptation automatique à la demande

2.5.2 Métriques de Performance

Le système monitore plusieurs indicateurs clés :

- Temps de réponse moyen ($< 500\text{ms}$ pour les opérations critiques)
- Taux d'erreur ($< 0.1\%$)
- Utilisation des ressources (CPU, mémoire)
- Débit de traitement (requêtes/seconde)

2.6 Déploiement et Orchestration

2.6.1 Pipeline CI/CD

Le système utilise GitLab CI/CD pour l'automatisation complète :

1. **Validation** : Tests de qualité et sécurité
2. **Construction** : Build des images Docker
3. **Tests** : Tests unitaires et d'intégration
4. **Déploiement** : Livraison en staging puis production

2.6.2 Environnements

- **Développement** : Environnement local avec Docker
- **Staging** : Environnement de pré-production
- **Production** : Environnement de production avec monitoring

2.7 Conclusion

L'architecture de CompanionIO combine les meilleures pratiques du développement moderne avec une approche centrée sur la scalabilité et la maintenabilité. Cette conception permet au système d'évoluer selon les besoins tout en maintenant des garanties élevées de disponibilité et de performance.

Le chapitre suivant détaille le service orchestrateur, cœur du système de coordination.

Chapter 3

Conversation Flow Architecture

This chapter provides a comprehensive overview of how CompanionIO orchestrates multi-turn conversations. It introduces the complete data flow, session lifecycle, and the coordination between core microservices without deep-diving into individual service details.

3.1 System Overview

CompanionIO is built around a conversational pipeline that transforms user speech into intelligent audio responses. The system operates as a coordinated orchestra in which the Orchestrator service conducts three primary services—STT (Speech-to-Text), LLM (Language Model), and TTS (Text-to-Speech)—each handling a distinct phase of the conversation.

The Orchestrator, running on port 8000, acts as the central conductor managing sessions, coordinating service communication, and maintaining conversation state. The STT service on port 8001 transcribes user audio streams into text using Azure Cognitive Services. The LLM service on port 8002 generates intelligent responses to user input via the Groq API. Finally, the TTS service on port 8003 converts those text responses into natural-sounding audio, also via Azure Cognitive Services.

3.2 End-to-End Conversation Flow

3.2.1 The Complete Conversation Lifecycle

[DIAGRAM PLACEHOLDER: Insert full conversation flow diagram here]

Expected content: Complete data flow from client connection through STT → LLM → TTS → audio response

A typical conversation moves through several distinct phases. First, the client establishes a WebSocket connection to the Orchestrator at `ws://host:8000/ws/session`. Upon connection, the Orchestrator initializes the session by creating a unique `SessionActor` for this connection. This actor then sets up connections to the STT, LLM, and TTS services, while session state—including conversation history and metadata—is stored in Redis.

Once the session is ready, the client begins sending raw audio bytes to the Orchestrator. The Orchestrator buffers these audio chunks and forwards them continuously to the STT service. The STT service processes the audio stream using Azure Speech Recognition,

sending back partial and final transcripts as they arrive. The Orchestrator collects these transcripts to form a complete user utterance.

When the actor detects the end of the user’s turn—either through silence timeout, an explicit signal, or a special marker—it sends a `TURN_END` marker to the client and updates the conversation history in Redis. The actor then

As the LLM tokens arrive, the TTS service begins synthesis immediately, streaming audio chunks back through the Orchestrator to the client, which plays the audio in real time. The client also receives descriptive text such as transcripts and responses. This cycle then repeats for the next user turn.

3.3 Session Management Architecture

3.3.1 The SessionActor Model

Each connected client is managed by a dedicated SessionActor—an independent task that owns the message queue for that session. The actor manages connections to the STT, LLM, and TTS services, maintains the conversation history (synchronized with Redis), coordinates the multi-service pipeline, and streams outputs to the client via an output queue.

[DIAGRAM PLACEHOLDER: SessionActor lifecycle and responsibilities]

Expected content: Actor initialization, running state, task coordination, cleanup on disconnect

3.3.2 State Externalization with Redis

CompanionIO implements Phase 1 state externalization using Redis for three primary purposes. Conversation history is stored as a Redis LIST under `history:session`.

Communication between the client and Orchestrator uses WebSocket with JSON for metadata and binary for audio. The client sends messages containing a `type` field—such as `audio_chunk`, `start_session`, or `stop_session`—along with binary audio data. The Orch

3.5.1 Handling Concurrent Streams

Inside each SessionActor, the Orchestrator manages multiple concurrent tasks running in parallel. These include receiving messages from the client, handling the STT streaming loop, processing LLM generation, and streaming TTS audio.

[DIAGRAM PLACEHOLDER: Task coordination inside SessionActor]

Expected content: Multiple async tasks (receive from client, STT loop, LLM processing, TTS streaming) running in parallel

Several key coordination points keep the pipeline flowing smoothly. The client sends audio continuously, and the Orchestrator buffers and forwards it to STT. As STT sends partial transcripts, the actor accumulates them into a full utterance. When the actor detects a turn boundary—via silence or a marker—it freezes input and sends the user message to the LLM. Once LLM tokens begin arriving, they are immediately sent to the client for text display and also buffered and sent to the TTS for audio synthesis. Importantly, TTS audio

chunks are streamed to the client while the LLM is still generating later tokens, enabling true parallelism.

3.5.2 Mailbox Pattern

The mailbox is a message queue that decouples the client WebSocket from actor processing. Messages from the client are queued in the mailbox, and the actor dequeues and processes them at its own pace. The system supports both an in-memory backend using AsyncIOQueue and a Redis-backed RedisMailbox. This design enables resilience because the mailbox persists even if the actor crashes.

3.6 Error Handling Resilience

3.6.1 Connection Failures

When a connection to STT, LLM, or TTS fails, the actor catches the error and notifies the client. If the client disconnects, the Orchestrator terminates the actor and frees all associated resources. In cases where a response is only partially completed, the client receives whatever was generated before the failure, ensuring at least some useful output.

3.6.2 Session Recovery (Phase 2 Future Plan)

With Redis persistence, future versions of CompanionIO will be able to recover conversation history after a pod restart, resume interrupted sessions on different pods, and implement session affinity in Kubernetes. This remains a planned enhancement for Phase 2.

3.7 Performance Characteristics

3.7.1 Latency

Typical end-to-end latency can be broken down into several phases. Audio capture for two to three seconds of user speech takes about two to three seconds. STT processing overlaps with speech and adds roughly 0.5 to 1 second. LLM token generation for twelve to twenty tokens takes two to four seconds, and TTS audio synthesis overlaps with LLM generation. The total perceived latency—from the end of user speech to the first audio response—typically falls between three and five seconds.

Phase	Typical Duration
2-3 seconds STT processing (streaming)	Audio capture (0.5-1 second overlap LLM token generation)
2-4 seconds TTS audio synthesis	Streaming, overlaps with LLM
3-5 seconds (from end of speech to first audio)	Total

Table 3.1: Typical conversation latency breakdown

3.7.2 Throughput

In terms of throughput, audio streaming runs at roughly 1600 bytes per chunk for 10ms chunks at 16kHz, which works out to about 160 kilobytes per second. LLM token generation is slower, typically one to three tokens per second depending on Groq rate limiting. TTS audio output is about 32 kilobytes per second for 16kHz mono 16-bit audio.

3.8 Key Design Decisions

Several fundamental design decisions shape the architecture. The system keeps each STT, LLM, and TTS service stateless so they can be restarted or scaled without side effects. Session state is externalized to Redis, which stores conversation history and enables fault tolerance. The practical implication is that the Orchestrator becomes the only stateful component per session.

The architecture is also streaming-first. Audio and LLM tokens are streamed rather than buffered, which lowers latency, improves real-time responsiveness, and reduces memory footprint. The trade-off is that it requires careful management of asynchronous tasks.

Finally, the system uses WebSocket for all inter-service communication. WebSocket was chosen over REST because it provides persistent, full-duplex connections with lower overhead for streaming. The implication is that the services are coupled at the protocol level, which could make swapping to gRPC more difficult in the future.

3.9 Chapter Summary

CompanionIO's conversation flow is a carefully orchestrated pipeline in which a SessionActor manages each client connection. Audio flows in, text passes through the LLM, and audio flows out. State is externalized to Redis for resilience. The services operate independently while the Orchestrator coordinates them, and the streaming design ensures low latency and a real-time feel.

The following chapters (4 through 7) dive deep into each service's implementation, infrastructure, and deployment considerations. Before reading those, you now understand why each service exists and what data flows between them.

Chapter 4

Orchestrator Service Implementation

This chapter documents the Orchestrator service as it exists in the current codebase. The Orchestrator is the central conductor of CompanionIO, responsible for managing client WebSocket connections, spawning session actors, coordinating the STT, LLM, and TTS services, and externalizing state to Redis. All descriptions in this chapter reflect the actual implementation; no speculative or planned features are included.

4.1 Service Overview

The Orchestrator runs on port 8000 and exposes three categories of endpoints: a WebSocket endpoint for real-time client communication, REST endpoints for session management, and a health check endpoint. The service is built with FastAPI and uses an actor-based concurrency model where each connected client receives its own dedicated SessionActor.

4.1.1 Entry Point: main.py

The FastAPI application is defined in main.py with a lifespan context manager that handles startup and shutdown. On startup, the Orchestrator connects to Redis via the supervisor. On shutdown, it terminates all active sessions and closes the Redis connection.

The logging configuration is set to INFO level with a standard format that includes timestamps, logger names, and log levels.

4.1.2 Global Supervisor Instance

The module initializes a single global SessionSupervisor instance. This supervisor manages all active sessions on the current pod. When the application receives a shutdown signal, it calls the supervisor's shutdown() method, which terminates all sessions in parallel and closes the Redis connection.

4.1.3 Health Check Endpoint

1 GET /health

Listing 4.1: GET /health

The /health endpoint returns a simple JSON response `{"status": "healthy"}` when the Orchestrator is running. This endpoint is used for Docker container health monitoring. It does not verify Redis connectivity.

4.1.4 Session Management REST Endpoints

Session management REST endpoints are defined in `session_router.py`. All endpoints depend on authentication via `get_user_context` imported from `main`.

POST /create

Creates a new session for the authenticated user. The session ID is constructed as `"session_{user_id}_{hash(str(user))}"`. The endpoint returns a JSON object containing the session ID.

GET /{session_id}

Retrieves information about a specific session. The endpoint verifies ownership by checking that the session ID starts with `"session_{user_id}"`. If the check fails, it returns a 403 HTTP exception.

DELETE /{session_id}

Deletes a session. Access is granted if the user is the session owner or if the user has the "Admin" role in their roles list. If neither condition is true, the endpoint returns a 403 HTTP exception.

GET /admin/all

An admin-only endpoint that returns all sessions. This endpoint uses the `require_admin` dependency imported from `main`. The current implementation returns a placeholder response containing an empty sessions list and a message indicating admin access was granted.

4.1.5 WebSocket Endpoint: /ws/session

The primary client-facing endpoint is defined in `main.py` as `@app.websocket("/ws/session")`. This endpoint accepts a WebSocket connection from a client, generates a unique UUID as the session ID, and spawns a `SessionActor` for that session.

The endpoint then runs two concurrent tasks:

- `receive_loop`: Continuously calls `ws.receive_bytes()` to receive audio chunks from the client. Each chunk is wrapped in a dictionary with type "audio" and placed into the actor's mailbox via `actor.send()`. On `WebSocketDisconnect` or any exception, it sends a "stop" message to the actor.
- `send_loop`: Continuously calls `actor.recv()` to receive output messages from the actor. Based on the message type, it sends data to the client:
 - "transcript": Sends text prefixed with "TRANSCRIPT: "

- "token": Sends the token as plain text
- "audio": Sends binary audio bytes via `ws.send_bytes()`
- "turn_complete": Sends the string `"__TURN_END__"`
- "error": Sends text prefixed with `"ERROR: "`
- "session_end": Breaks the loop and exits

Both tasks run concurrently using `asyncio.gather()`. When the gather completes, the endpoint cancels both tasks and calls `supervisor.terminate(session_id)` to clean up resources.

4.2 Session Supervisor

The `SessionSupervisor` class in `supervisor.py` is the master orchestrator for all active sessions on a single pod. It maintains an in-memory dictionary `self._actors` mapping session IDs to `SessionActor` instances.

4.2.1 Initialization

The constructor initializes an empty actor registry and sets `self.redis_client` to `None`. The Redis client is not created in the constructor because Redis connection requires async operations, which are handled in the lifespan startup.

4.2.2 Redis Connection

The `connect_redis()` method creates a `RedisClient` instance using `settings.redis_url` and calls `await self.redis_client.connect()`. This method is called from the FastAPI lifespan on startup. If Redis connection fails, the error is logged and the supervisor continues with in-memory mailbox functionality.

4.2.3 Spawning an Actor

The `spawn(session_id)` method creates a new session actor. The steps are:

1. Create `STTClient`, `LLMClient`, and `TTSCClient` instances using the URLs from `settings`.
2. Call `connect()` on each client to establish WebSocket connections to the respective services.
3. Create a `SessionActor` instance, passing the session ID, the three service clients, and the Redis client.
4. Call `await actor.start()` to launch the actor's background tasks.
5. Store the actor in `self._actors[session_id]`.
6. Call `await self.redis_client.set_session_metadata()` with the pod name (obtained from `socket.gethostname()`), the current UTC timestamp, and status `"active"`.

If any service client fails to connect, the exception is logged and re-raised, preventing actor creation.

4.2.4 Retrieving an Actor

The `get(session_id)` method performs a local lookup in `self._actors` and returns the actor if found, otherwise returns `None`. This method does not query Redis; it only returns actors running on the current pod.

4.2.5 Terminating an Actor

The `terminate(session_id)` method:

1. Pops the actor from `self._actors` using `dict.pop()`. If no actor is found, it logs a warning and returns.
2. Calls `await actor.stop()` to cancel the actor's main task.
3. Calls `close()` on the STT, LLM, and TTS clients.
4. Calls `await self.redis_client.clear_session(session_id)` to delete all Redis data for this session.

4.2.6 Active Sessions Count

The `active_sessions` property returns `len(self._actors)`.

4.2.7 Shutdown

The `shutdown()` method:

1. Creates a list of termination tasks for all session IDs in `self._actors`.
2. Runs these tasks in parallel using `asyncio.gather()` with `return_exceptions=True`.
3. If a Redis client exists, calls `await self.redis_client.close()`.

4.3 Session Actor

The `SessionActor` class in `actor.py` manages one user's conversation. Each actor owns its session state, coordinates the STT → LLM → TTS pipeline, and handles the mailbox and output queue.

4.3.1 Initialization

The constructor accepts:

- `session_id`: Unique session identifier (UUID string)
- `stt`: Connected `STTClient` instance

- `llm`: Connected `LLMClient` instance
- `tts`: Connected `TTSCClient` instance
- `redis_client`: Optional `RedisClient` instance

The actor initializes the following instance variables:

- `self.history`: An empty list that will store conversation messages as dictionaries with `role` and `content` keys.
- `self.mailbox`: If `redis_client` is provided, creates a `RedisMailbox` instance; otherwise creates an in-process `Mailbox` instance.
- `self.output_queue`: An `asyncio.Queue` for messages to be sent to the client.
- `self._task` and `self._stt_task`: Initially `None`; these will hold the background task objects.
- `self.full_transcript`: An empty string that accumulates STT text until the turn ends.

4.3.2 Starting the Actor

The `start()` method creates two background tasks:

1. `self._stt_task = asyncio.create_task(self._stt_loop())`
2. `self._task = asyncio.create_task(self._run())`

4.3.3 Sending Messages to the Actor

The `send(msg)` method calls `await self.mailbox.enqueue(msg)`. Message types expected by the actor include `"audio"`, `"turn_end"`, and `"stop"`.

4.3.4 Receiving Messages from the Actor

The `recv()` method calls `await self.output_queue.get()`. This blocks until a message is available, then returns the message.

4.3.5 Main Loop: `_run()`

The main loop runs continuously until a `"stop"` message is received. It dequeues messages from the mailbox and handles each type:

- `"heartbeat"`: Ignored and continues.
- `"audio"`: Calls `self._handle_audio(msg["data"])`
- `"turn_end"`: Calls `self._handle_turn_end()`
- `"stop"`: Calls `self._handle_close()` and breaks the loop.

Any exception in the loop causes it to break and log the error.

4.3.6 STT Loop: `_stt_loop()`

The STT loop runs continuously and polls the STT client for transcripts. For each transcript received:

1. Strips whitespace and stores the transcript in `self.full_transcript`.
2. Appends a message dictionary `{"role": "user", "content": transcript}` to `self.history`.
3. If a Redis client exists, calls `await self.redis_client.append_to_history()` with the same message.
4. Puts a `{"type": "transcript", "data": transcript}` message into the output queue.
5. Creates a background task calling `self._handle_turn_end(self.full_transcript)` using `asyncio.create_task()`. This is non-blocking; the STT loop continues immediately.

If `stt.recv_text()` returns `None`, the loop continues without processing.

4.3.7 Turn Processing: `_handle_turn_end(prompt)`

This method processes the end of a user's utterance. It accepts an optional `prompt` parameter that defaults to `None`. If `prompt` is `None`, it uses `self.full_transcript`. If the resulting prompt is empty, the method returns early.

The processing steps are:

1. Calls `self.llm.generate(prompt)` which returns an async generator yielding tokens.
2. Iterates over each token, appends it to `full_response`, and puts a `{"type": "token", "data": token}` message into the output queue.
3. If the final `full_response` is not empty, calls `self.tts.synthesize(full_response)` which returns an async generator yielding audio chunks.
4. Iterates over each audio chunk and puts a `{"type": "audio", "data": chunk}` message into the output queue.
5. Appends an assistant message `{"role": "assistant", "content": full_response}` to `self.history`.
6. If a Redis client exists, appends the same message to Redis history.
7. Puts a `{"type": "turn_complete"}` message into the output queue.

If an exception occurs during processing, the method puts an `{"type": "error", "data": str(e)}` message into the output queue.

In the finally block, if the `prompt` parameter equals the current `self.full_transcript` value, it resets `self.full_transcript` to an empty string.

4.3.8 Audio Handling: `_handle_audio(audio)`

This method calls `await self.stt.send_audio(audio)` to forward the audio chunk to the STT service.

4.3.9 Close Handling: `_handle_close()`

This method handles session closure:

1. Cancels the STT loop task if it exists.
2. If `self.full_transcript.strip()` is not empty, calls `self._handle_turn_end(self.full_transcript)` to process the final pending transcript.
3. Puts a `{"type": "session_end"}` message into the output queue.

4.3.10 Stopping the Actor

The `stop()` method cancels the main task `self._task` if it exists. It catches the `CancelledError` and logs debug information.

4.4 Redis Client

The `RedisClient` class in `redis_client.py` provides async Redis operations for three data structures.

4.4.1 Connection Management

The constructor accepts a `redis_url` string and initializes `self._client` as `None`. The `connect()` method creates a Redis connection using `redis.asyncio.from_url()` with decoding enabled and a 30-second health check interval. It verifies the connection by calling `ping()`. The `close()` method closes the connection.

4.4.2 Conversation History (Redis LIST)

- Key format: `history:{session_id}`
- Expiration: 86400 seconds (24 hours)

The `append_to_history(session_id, message)` method serializes the message dictionary to JSON using `json.dumps()`, calls `self._client.rpush(key, message_json)`, and sets the key expiration. It returns the length of the list after the push.

The `get_history(session_id)` method calls `self._client.lrange(key, 0, -1)` to retrieve all messages, deserializes each from JSON, and returns the list. On error, it returns an empty list.

The `clear_history(session_id)` method deletes the key.

4.4.3 Mailbox (Redis STREAM)

- Key format: `mailbox:{session_id}`
- Expiration: 86400 seconds (24 hours)

The `enqueue_message(session_id, message)` method serializes the message to JSON using `json.dumps(message, default=str)` (the `default=str` handler allows serialization of bytes). It calls `self._client.xadd(key, {"msg": message_json})` and returns the message ID.

The `dequeue_message(session_id, last_id, timeout)` method calls `self._client.xread([last_id], block=timeout, count=1)`. If the result contains messages, it extracts the message ID and fields, deserializes the "msg" field from JSON, and returns a tuple of (msg_id, message). On timeout or error, it returns None.

The `mailbox_len(session_id)` method calls `self._client.xlen(key)`.

The `clear_mailbox(session_id)` method deletes the key.

4.4.4 Session Metadata (Redis HASH)

- Key format: `session:{session_id}`
- Expiration: 86400 seconds (24 hours)

The `set_session_metadata(session_id, metadata)` method calls `self._client.hset(key, mapping=metadata)` and sets the key expiration.

The `get_session_metadata(session_id)` method calls `self._client.hgetall(key)` and returns the result or an empty dictionary.

4.4.5 Session Cleanup

The `clear_session(session_id)` method calls `self._client.keys(f"*{session_id}")` to find all keys containing the session ID, then deletes them using `self._client.delete()`.

4.4.6 Health Check

The `health_check()` method calls `self._client.ping()` and returns True on success, False on exception.

4.5 Mailbox Abstraction

The `mailbox.py` module provides a queue abstraction that decouples the actor from the underlying queue implementation.

4.5.1 Mailbox Class

The Mailbox class is a wrapper that delegates to an internal implementation. Its constructor sets `self._impl = AsyncIOQueue()`. It provides `enqueue(msg)`, `dequeue()`, and `empty()` methods that forward to the implementation.

4.5.2 AsyncIOQueue Class

This class wraps `asyncio.Queue`. The `enqueue(msg)` method calls `await self._q.put(msg)`. The `dequeue()` method calls `await self._q.get()`. The `empty()` method returns `self._q.empty()`.

4.5.3 RedisMailbox Class

This class implements a Redis Streams-based mailbox. The constructor accepts a `redis_client` and `session_id`, and initializes `self.last_id = "0"`.

The `enqueue(msg)` method calls `await self.redis_client.enqueue_message(self.session_id, msg)`.

The `dequeue()` method calls `await self.redis_client.dequeue_message(self.session_id, last_id=self.last_id, timeout=1000)`. If a message is returned, it updates `self.last_id = msg_id` and returns the message. If no message is returned (timeout or error), it returns `{"type": "heartbeat"}`.

The `empty()` method calls `await self.redis_client.mailbox_len(self.session_id)` and returns `True` if the length is zero.

4.6 Configuration

The `Settings` class in `settings.py` uses `pydantic_settings.BaseSettings` to load configuration from environment variables.

4.6.1 Configuration Variables

- `stt_url`: Default `"ws://localhost:8001/stt"`. Override with `STT_URL` environment variable.
- `llm_url`: Default `"ws://localhost:8002/llm"`. Override with `LLM_URL` environment variable.
- `tts_url`: Default `"ws://localhost:8003/tts"`. Override with `TTS_URL` environment variable.
- `redis_url`: Default `"redis://localhost:6379"`. Override with `REDIS_URL` environment variable.

A global settings instance is created at module level.

4.7 Service Clients

4.7.1 STTClient

The `STTClient` class in `stt_client.py` manages the WebSocket connection to the STT service.

- `connect()`: Establishes a WebSocket connection to `self.url`.
- `send_audio(chunk)`: Sends raw audio bytes over the WebSocket.

- `recv_text()`: Receives a message from the WebSocket. Returns the received text or None on exception.
- `close()`: Closes the WebSocket connection if it exists.

4.7.2 LLMClient

The LLMClient class in `llm_client.py` manages the WebSocket connection to the LLM service.

The `_build_system_prompt()` method returns a string containing the assistant's identity, capabilities, mission, and guardrails.

The `connect()` method establishes a WebSocket connection to `self.url`.

The `generate(prompt)` method is an async generator. It constructs `full_prompt` by concatenating the system prompt, "User: ", the user prompt, and "\nAssistant:". It sends this string over the WebSocket. Then it iterates over messages received from the WebSocket, yielding each token until a message equal to "<END>" is received, at which point the generator stops.

The `close()` method closes the WebSocket connection if it exists.

4.7.3 TTSClient

The TTSClient class in `tts_client.py` manages the WebSocket connection to the TTS service.

The `connect()` method establishes a WebSocket connection to `self.url`.

The `synthesize(text)` method is an async generator. It sends the input text over the WebSocket, then enters a loop receiving messages. For each received chunk, it yields the chunk. The loop continues until a `asyncio.TimeoutError` occurs (timeout set to 2.0 seconds) or another exception is caught.

The `close()` method closes the WebSocket connection if it exists.

4.8 WebSocket Router (Partial Implementation)

The file `websocket_router.py` contains a router with an endpoint `/session/{session_id}` that includes authentication via `get_user_context`. This endpoint accepts the WebSocket, verifies that the session ID starts with `f"session_{user['user_id']}"`, and then echoes received text back to the client. This file appears to be a separate or older implementation; the primary WebSocket endpoint used by the Orchestrator is `/ws/session` in `main.py`, not this router. The file `websocket.py` similarly appears to be a different implementation using a `Pipeline` class that is not defined in the provided code. These files are documented as they exist but are not part of the active Orchestrator implementation described in `main.py`.

4.9 Summary of External Dependencies

The Orchestrator service depends on the following external libraries:

- `fastapi`: Web framework

- `uvicorn`: ASGI server (not shown in code but implied by FastAPI usage)
- `websockets`: For client WebSocket connections to STT, LLM, and TTS services
- `redis.asyncio`: For Redis client operations
- `pydantic_settings`: For configuration management
- `asyncio`: For concurrent task management

4.10 Summary

The Orchestrator service as implemented provides:

- A WebSocket endpoint at `/ws/session` that accepts client connections and spawns per-session actors.
- REST endpoints for session creation, retrieval, deletion, and admin listing.
- A `SessionSupervisor` that manages all active actors on a single pod.
- A `SessionActor` per client that coordinates the STT → LLM → TTS pipeline.
- Redis integration for conversation history, mailbox streams, and session metadata.
- Mailbox abstraction supporting both in-memory and Redis-backed queues.
- Service clients for STT, LLM, and TTS that use WebSocket protocols.

The current implementation assumes all three backend services (STT, LLM, TTS) are available at the URLs specified in settings. If Redis is unavailable, the Orchestrator logs a warning and continues using in-memory mailboxes. No automatic session recovery across pod restarts is implemented in the current code.

4.11 STT Service Implementation

The STT (Speech-to-Text) service transcribes user audio streams into text using Azure Cognitive Services. The service exposes a WebSocket endpoint that accepts binary audio chunks and streams back transcribed text in real time.

4.11.1 Service Overview

The STT service runs on port 8001 and is built with FastAPI. It consists of two files: `main.py` containing the WebSocket endpoint and application logic, and `speech_service.py` containing the Azure Speech SDK wrapper. The service loads Azure credentials from environment variables using `python-dotenv`.

4.11.2 Entry Point: main.py

The FastAPI application initializes a global `SpeechService` instance. The service expects two environment variables to be loaded via `load_dotenv()`: `SPEECH_KEY` and `SPEECH_REGION`.

4.11.3 Health Check Endpoint

```
1 GET /health
```

Listing 4.2: GET /health

The `/health` endpoint returns a JSON response indicating the service is healthy:

```
1 {"status": "healthy", "service": "stt"}
```

4.11.4 WebSocket Endpoint: /stt

The primary endpoint is `@app.websocket("/stt")`. When a client connects, the endpoint accepts the `WebSocket` and sets up continuous speech recognition using `Azure`.

Connection Flow

Upon `WebSocket` connection, the endpoint:

1. Accepts the `WebSocket` with `await ws.accept()`
2. Prints `"[STT] Client connected"`
3. Retrieves the running `asyncio` event loop via `asyncio.get_running_loop()`
4. Creates an `asyncio.Queue` named `text_queue` to hold recognized text
5. Creates a recognizer and push audio stream via `speech_service.create_recognizer()`
6. Defines a recognized callback function for `Azure` recognition events
7. Connects the callback to `recognizer.recognized`
8. Starts continuous recognition with `recognizer.start_continuous_recognition()`
9. Creates two concurrent tasks: `receive_audio()` and `send_transcripts()`
10. Runs both tasks with `asyncio.gather()`

Recognized Callback Function

The recognized function is called by `Azure` when speech is recognized:

```
1 def recognized(evt):  
2     if evt.result.text:  
3         print(f"[STT] {Azure} recognized: '{evt.result.text}'")  
4         loop.call_soon_threadsafe(text_queue.put_nowait, evt.result.text  
    )
```

The callback:

- Checks that `evt.result.text` is non-empty
- Prints the recognized text with a marker
- Uses `loop.call_soon_threadsafe()` to schedule `text_queue.put_nowait()` on the asyncio event loop

The use of `call_soon_threadsafe` is necessary because Azure's recognition callbacks run from a background thread, not the asyncio event loop thread.

Audio Reception Task: `receive_audio()`

This asynchronous task runs a loop that continuously receives binary audio chunks from the client:

1. Calls `await ws.receive_bytes()` to get the next audio chunk
2. Prints the size of the received chunk: `"[STT] Received {len(audio_chunk)} bytes"`
3. Writes the chunk to the Azure push audio stream using `stream.write(audio_chunk)`

The loop continues until an exception occurs (typically when the client disconnects), at which point it prints `"[STT] Audio receive stopped: {e}"` and exits.

Transcript Sending Task: `send_transcripts()`

This asynchronous task runs a loop that sends recognized text to the client:

1. Calls `await text_queue.get()` to wait for the next recognized text
2. Prints `"[STT] Sending transcript: '{text}'"`
3. Sends the text to the client via `await ws.send_text(text)`

The loop continues until an exception occurs, at which point it prints `"[STT] Transcript send stopped: {e}"` and exits.

Cleanup

In the `finally` block, the endpoint performs cleanup:

1. Cancels both `receive_task` and `send_task`
2. Closes the push audio stream with `stream.close()`
3. Stops continuous recognition with `recognizer.stop_continuous_recognition()`
4. Prints `"[STT] Cleaned up"`

4.11.5 Speech Service Class: `speech_service.py`

The `SpeechService` class provides a single method `create_recognizer()`.

Environment Variables

The method retrieves two environment variables:

- `SPEECH_KEY`: Azure Cognitive Services API key
- `SPEECH_REGION`: Azure region (e.g., "eastus")

If either variable is missing, the method raises a `ValueError` with the message "I key mafamesh aysh khouya" (Arabic colloquial for "the key doesn't exist, my brother").

Speech Configuration

The method creates a `speechsdk.SpeechConfig` object with the subscription key and region.

Push Audio Stream

The method creates a `speechsdk.audio.PushAudioInputStream`. This stream allows the service to push audio data from the `WebSocket` into Azure's recognition engine.

Audio Configuration

An `AudioConfig` is created with the push stream, connecting the stream to the recognizer.

Recognizer Creation

The method creates a `speechsdk.SpeechRecognizer` with the speech configuration and audio configuration, then returns a tuple containing the recognizer and the push stream:

```
1 return recognizer, stream
```

4.11.6 Error Handling

The `WebSocket` endpoint wraps the `asyncio.gather()` call in a try-except block:

- On exception during gathering, prints "[STT] Connection closed: {e}"
- Cleanup occurs in the finally block regardless of whether an exception occurred

Individual tasks handle their own exceptions and exit gracefully without crashing the entire endpoint.

4.11.7 Logging

The STT service uses `print()` statements for logging. The following log messages are produced:

- "[STT] Client connected" - When a client establishes a WebSocket connection
- "[STT] Azure recognized: '{text}'" - When Azure returns a recognized transcript
- "[STT] Received {len(audio_chunk)} bytes" - For each audio chunk received from the client
- "[STT] Sending transcript: '{text}'" - Before sending a transcript to the client
- "[STT] Audio receive stopped: {e}" - When the audio reception task ends
- "[STT] Transcript send stopped: {e}" - When the transcript sending task ends
- "[STT] Connection closed: {e}" - On exception during `asyncio.gather()`
- "[STT] Cleaned up" - After cleanup completes

4.11.8 External Dependencies

The STT service depends on:

- `fastapi`: Web framework
- `azure.cognitiveservices.speech`: Azure Speech SDK for speech recognition
- `python-dotenv`: Loading environment variables from `.env` files
- `asyncio`: Concurrent task management

4.11.9 Summary

The STT service as implemented provides:

- A WebSocket endpoint at `/stt` that accepts binary audio chunks and streams back transcribed text
- Integration with Azure Cognitive Services using the Speech SDK
- Push audio streaming via `PushAudioInputStream`
- Continuous recognition mode that processes audio as it arrives
- Thread-safe communication between Azure's callback thread and the `asyncio` event loop using `call_soon_threadsafe`
- Two concurrent tasks: one for receiving audio, one for sending transcripts

The service requires the `SPEECH_KEY` and `SPEECH_REGION` environment variables to be set. All logging is done via `print()` statements. The service does not perform any audio format validation; it assumes the client sends audio in a format compatible with Azure (typically 16kHz, mono, PCM).

4.12 LLM Service Implementation

The LLM (Language Model) service generates intelligent responses to user input using Groq's API. The service exposes a WebSocket endpoint that accepts text prompts and streams back response tokens in real time, terminating with a special marker.

4.12.1 Service Overview

The LLM service runs on port 8002 and is built with FastAPI. It consists of two files: `main.py` containing the WebSocket endpoint and application logic, and `llm_service.py` containing the Groq API wrapper. The service loads the Groq API key from environment variables using `python-dotenv`.

4.12.2 Entry Point: `main.py`

The FastAPI application initializes a global `LLMService` instance. The service expects the `GROQ_API_KEY` environment variable to be set.

4.12.3 Health Check Endpoint

```
1 GET /health
```

Listing 4.3: GET `/health`

The `/health` endpoint returns a JSON response indicating the service is healthy:

```
1 {"status": "healthy", "service": "llm"}
```

4.12.4 WebSocket Endpoint: `/llm`

The primary endpoint is `@app.websocket("/llm")`. When a client connects, the endpoint accepts the WebSocket and enters a loop that processes incoming text prompts.

Connection Flow

Upon WebSocket connection, the endpoint:

1. Accepts the WebSocket with `await ws.accept()`
2. Prints "LLM connected"
3. Enters a try-except block with a `while True` loop

Prompt Reception Loop

The main loop receives text prompts from the client using `await ws.receive_text()`. For each prompt received:

1. Prints "[LLM PROMPT]:" followed by the prompt
2. Prints "[LLM RESPONSE]: " without a newline, with `flush=True`
3. Calls `llm_service.stream_completion(prompt)` which returns a generator yielding tokens
4. Iterates over each token, sends it to the client via `await ws.send_text(token)`, and prints the token without a newline
5. After all tokens have been streamed, sends "<END>" to the client to mark the end of the response

Error Handling

The loop wraps the prompt processing in a try-except block. On exception, the endpoint prints "LLM closed:" followed by the exception message. No reconnection or retry logic is implemented.

4.12.5 LLM Service Class: `llm_service.py`

The `LLMService` class encapsulates the Groq API client and provides a streaming completion method.

Initialization

The constructor retrieves the Groq API key from environment variables:

```
1 api_key = os.getenv("GROQ_API_KEY")
2 if not api_key:
3     raise ValueError("Missing GROQ_API_KEY")
```

If the environment variable is not set, the constructor raises a `ValueError` with the message "Missing GROQ_API_KEY".

The constructor then initializes the Groq client:

```
1 self.client = Groq(api_key=api_key)
```

Stream Completion Method

The `stream_completion(prompt)` method accepts a prompt string and returns a generator that yields tokens one by one.

API Parameters The method calls `self.client.chat.completions.create()` with the following parameters:

- `model`: Hardcoded to "llama-3.3-70b-versatile"
- `messages`: A list containing a single message dictionary with role "user" and content set to the prompt argument
- `temperature`: Hardcoded to 0.7
- `stream`: Set to True to enable token-by-token streaming

Token Streaming The method iterates over the completion chunks:

```
1 for chunk in completion:
2     token = chunk.choices[0].delta.content
3     if token:
4         yield token
```

For each chunk:

- Extracts `chunk.choices[0].delta.content` which contains the next token
- If the token is not None (not all chunks contain content), yields the token
- Tokens that are None (e.g., the final chunk with no content) are silently skipped

4.12.6 Protocol

Client to Service

The client sends plain text prompts. No JSON wrapping is used. Each prompt is sent as a single WebSocket text message.

Service to Client

The service sends:

- Individual tokens as plain text WebSocket messages
- A final "<END>" message after all tokens have been sent

The "<END>" marker is sent unconditionally after every prompt, even if the response was empty.

4.12.7 Logging

The LLM service uses `print()` statements for logging. The following log messages are produced:

- "LLM connected" - When a client establishes a WebSocket connection

- `"\n[LLM PROMPT]: {prompt}"` - When a prompt is received
- `"[LLM RESPONSE]: "` - Printed before streaming begins, with `end=""` and `flush=True`
- Each token printed as it is streamed, with `end=""` and `flush=True`
- `"LLM closed: {e}"` - When an exception occurs during processing

Note that no newline is printed after the response stream completes.

4.12.8 External Dependencies

The LLM service depends on:

- `fastapi`: Web framework
- `groq`: Groq API client library
- `python-dotenv`: Loading environment variables from `.env` files

4.12.9 Configuration

The service uses the following hardcoded configuration values:

- Model: `llama-3.3-70b-versatile`
- Temperature: `0.7`
- Streaming: `Enabled`

No environment variable is provided to override the model or temperature. The only required environment variable is `GROQ_API_KEY`.

4.12.10 Summary

The LLM service as implemented provides:

- A WebSocket endpoint at `/llm` that accepts text prompts and streams back token responses
- Integration with Groq's API using the `llama-3.3-70b-versatile` model
- Token-by-token streaming with a `"<END>"` marker to signal response completion
- No conversation history or system prompt management-each request is independent and receives only the user's prompt
- No prompt preprocessing; the prompt is sent directly to Groq as-is

The service requires the `GROQ_API_KEY` environment variable to be set. Each WebSocket connection can handle multiple sequential prompts; the connection remains open until the client disconnects or an error occurs. The service does not include authentication, rate limiting, or request validation beyond basic WebSocket reception.

4.13 TTS Service Implementation

The TTS (Text-to-Speech) service converts text responses into natural-sounding audio using Azure Cognitive Services. The service exposes a WebSocket endpoint that accepts text and streams back audio chunks in real time.

4.13.1 Service Overview

The TTS service runs on port 8003 and is built with FastAPI. It consists of two files: `main.py` containing the WebSocket endpoint and application logic, and `tts_service.py` containing the Azure Speech SDK wrapper. The service loads Azure credentials from environment variables using `python-dotenv`.

4.13.2 Entry Point: `main.py`

The FastAPI application initializes a global `TTSService` instance. The service expects two environment variables to be loaded via `load_dotenv()`: `SPEECH_KEY` and `SPEECH_REGION`. If these are missing, the service will raise an error when attempting to create a synthesizer.

4.13.3 Health Check Endpoint

```
1 GET /health
```

Listing 4.4: GET `/health`

The `/health` endpoint returns a JSON response indicating the service is healthy:

```
1 {"status": "healthy", "service": "tts"}
```

4.13.4 WebSocket Endpoint: `/tts`

The primary endpoint is `@app.websocket("/tts")`. When a client connects, the endpoint accepts the WebSocket and enters a loop that processes incoming text messages.

Connection Flow

Upon WebSocket connection, the endpoint:

1. Accepts the WebSocket with `await ws.accept()`
2. Prints `"[TTS] client connected"`
3. Retrieves the running `asyncio` event loop
4. Enters the main processing loop

Text Reception Loop

The main loop receives text messages from the client using `await ws.receive_text()`. For each text message received:

1. Prints the received text
2. Creates an `asyncio.Queue` with `maxsize=20` to hold audio chunks
3. Initializes an empty `audio_buffer` as a `bytearray`
4. Defines an `AudioStreamCallback` class (see below)
5. Creates a synthesizer via `tts_service.create_synthesizer(audio_callback)`
6. Launches a background synthesis task using `asyncio.create_task()` that runs `speak_text_async(text).get()` in a thread via `asyncio.to_thread()`
7. Enters a streaming loop to send audio chunks to the client
8. After synthesis completes, writes the accumulated audio buffer to a WAV file

AudioStreamCallback Class

The `AudioStreamCallback` class is defined inside the WebSocket handler. It provides a `write` method that Azure's `PushAudioOutputStream` calls when audio data is available:

- `__init__(self, queue, loop)`: Stores the `asyncio` queue and event loop.
- `write(self, data: memoryview)`: Converts the `memoryview` to bytes, prints the chunk size, schedules the chunk to be put into the queue using `asyncio.run_coroutine_threadsafe()` and returns the length of the data.
- `close(self)`: Prints a debug message when the stream closes.

Streaming Loop

The streaming loop continues until both conditions are met: the synthesis task is done AND the audio queue is empty. Inside the loop:

1. Attempts to get a chunk from the queue with `asyncio.wait_for(audio_queue.get(), timeout=0.5)`.
2. If a `TimeoutError` occurs, the loop continues.
3. When a chunk is received, it is appended to `audio_buffer` and sent to the client via `await ws.send_bytes(chunk)`.
4. If sending fails (WebSocket closed), the loop breaks.

The timeout mechanism prevents the loop from blocking indefinitely if the queue becomes empty before synthesis completes.

WAV File Output

After the streaming loop exits and synthesis completes, the endpoint writes the accumulated audio buffer to a WAV file:

- Opens a file named `tts_out_{timestamp}.wav` using `wave.open()`
- Sets channel count to 1 (mono)
- Sets sample width to 2 (16-bit)
- Sets frame rate to 16000 Hz
- Writes the audio buffer frames

The timestamp is obtained from `loop.time()` cast to integer. If the audio buffer is empty, the service prints a warning and does not write a file.

4.13.5 TTS Service Class: `tts_service.py`

The `TTSService` class provides a single method `create_synthesizer(audio_callback)`.

Environment Variables

The method retrieves two environment variables:

- `SPEECH_KEY`: Azure Cognitive Services API key
- `SPEECH_REGION`: Azure region (e.g., "eastus")

If either variable is missing, the method raises a `ValueError` with the message "Missing Azure Speech credentials".

Speech Configuration

The method creates a `speechsdk.SpeechConfig` object with the subscription key and region.

The voice is hardcoded to "en-US-JennyNeural". No configuration option is provided in the current code to change the voice.

Audio Output Configuration

The method creates a `speechsdk.audio.PushAudioOutputStream` using the provided `audio_callback`. This stream allows Azure to push audio data to the callback as it becomes available, rather than writing to a file or speaker.

An `AudioOutputConfig` is created with `stream=stream` to route synthesized audio to the push stream.

Synthesizer Creation

The method creates and returns a `speechsdk.SpeechSynthesizer` with the speech configuration and audio output configuration.

4.13.6 Error Handling

The WebSocket endpoint wraps the entire processing loop in a try-except-finally block:

- On exception, prints "[TTS] error: " followed by the exception message.
- In the finally block, prints "[TTS] client disconnected".

If the WebSocket closes during audio streaming, the exception is caught and the loop breaks. The synthesis task is still awaited to ensure it completes.

4.13.7 Logging

The TTS service uses `print()` statements for logging rather than the logging module. The following log messages are produced:

- "[TTS] client connected" - When a client establishes a WebSocket connection
- "[TTS] received text: {text}" - When text is received from the client
- "[TTS CALLBACK] chunk: {len(chunk)}" - For each audio chunk from Azure
- "[TTS] synthesis started" - Before the streaming loop begins
- "[TTS] synthesis finished (task done + queue empty)" - When synthesis completes normally
- "[TTS] websocket closed during stream: {e}" - If the WebSocket closes while sending
- "[TTS] wrote tts_out.wav" - After successfully writing the WAV file
- "[TTS] empty audio buffer, no file written" - If no audio data was received
- "[TTS] error: {e}" - On any exception during processing
- "[TTS] client disconnected" - When the connection ends

4.13.8 External Dependencies

The TTS service depends on:

- `fastapi`: Web framework
- `azure.cognitiveservices.speech`: Azure Speech SDK for text-to-speech synthesis
- `python-dotenv`: Loading environment variables from `.env` files
- `asyncio`: Concurrent task management
- `wave`: WAV file writing (standard library)

4.13.9 Summary

The TTS service as implemented provides:

- A WebSocket endpoint at /tts that accepts text and streams audio chunks
- Integration with Azure Cognitive Services using the Speech SDK
- Push audio streaming via PushAudioOutputStream and a callback class
- Asynchronous coordination between the Azure callback (which runs in a thread) and the asyncio event loop
- A timeout-based streaming loop that continues until synthesis completes and the queue is empty
- Debug output that writes received audio to timestamped WAV files

The service uses a hardcoded voice (en-US-JennyNeural) and requires the SPEECH_KEY and SPEECH_REGION environment variables to be set. All logging is done via print() statements. The service writes WAV files to the current working directory for each synthesis operation.

Chapter 5

Supporting Services

This chapter documents the supporting services that complement the core conversation pipeline. These services provide authentication, authorization, and service-account management for the CompanionIO ecosystem.

5.1 Auth Service

The Auth service provides user authentication and service-account (agent) management using ASP.NET Core Identity, JWT tokens, and Google OAuth. The service is built on .NET 8 and uses Entity Framework Core with PostgreSQL (with SQLite fallback for development).

5.1.1 Service Overview

The Auth service runs as a .NET 8 Web API. It handles user registration, login, Google OAuth integration, JWT issuance and refresh, and agent (service account) management. The service integrates with the Orchestrator's authentication dependencies, which expect user context from this service.

5.1.2 Project Structure

The service root is `Services/Auth`. The key files are:

- `Program.cs`: Application startup, environment variable loading, database registration, Identity and authentication configuration
- `Controllers/AuthController.cs`: User registration, login, Google OAuth flows, and user info endpoints
- `Controllers/AgentsController.cs`: Agent (service account) registration and authentication, restricted to Admin role
- `Data/AuthDbContext.cs`: Identity DbContext with seeded roles
- `Models/ApplicationUser.cs`: Custom Identity user model extending `IdentityUser`
- `Migrations/`: EF Core migrations that create Identity tables
- `appsettings.json`: Default configuration for JWT, Google OAuth, and connection strings

5.1.3 Environment Variables and Configuration

Environment Variable Loading

The `Program.cs` file includes a function `FindEnvFile()` that searches for a `.env` file. It looks up to five parent directories from both the current working directory and `AppContext.BaseDirectory`. When found, it loads the file using `DotNetEnv.Env.Load()`.

Database Connection Resolution

The service determines the database connection as follows:

1. If `PGHOST` environment variable is set, the service constructs a PostgreSQL connection string using:

- `PGHOST` (required)
- `PGPORT` (default: "5432")
- `PGUSER` (default: "postgres")
- `PGPASSWORD` (default: empty string)
- `PGDATABASE` (default: "authdb")

The connection string includes `Ssl Mode=Require;Trust Server Certificate=true`. The service then uses `Npgsql` as the database provider.

2. If `PGHOST` is not set and the environment is `Development`, the service uses `SQLite` with a database file at `auth_dev.db`.
3. Otherwise, the service falls back to the connection string from `appsettings.json` under `ConnectionStrings:DefaultConnection`.

Required Environment Variables

The following environment variables must be set for production operation:

- `PGHOST`, `PGUSER`, `PGPASSWORD`, `PGDATABASE` (for PostgreSQL)
- `Jwt:Key`, `Jwt:Issuer`, `Jwt:Audience` (loaded from `appsettings.json` or environment)
- `Google:ClientId`, `Google:ClientSecret` (for Google OAuth, if used)

5.1.4 Database Schema

ApplicationUser Model

The `ApplicationUser` class extends `IdentityUser` and adds two additional properties:

- `FullName`: A nullable string property for the user's full name
- `CreatedAt`: A `DateTime` property set to `DateTime.UtcNow` upon user creation

Identity Tables

The Entity Framework Core migration creates the standard ASP.NET Identity tables:

- **AspNetUsers:** Stores user accounts with columns including Id, FullName, CreatedAt, UserName, NormalizedUserName, Email, NormalizedEmail, EmailConfirmed, PasswordHash, and other Identity standard fields
- **AspNetRoles:** Stores roles with columns Id, Name, NormalizedName
- **AspNetUserRoles:** Junction table mapping users to roles
- **AspNetUserClaims, AspNetUserLogins, AspNetUserTokens:** Identity user-related tables
- **AspNetRoleClaims:** Role claims table
- **__EFMigrationsHistory:** Tracks applied migrations

Seeded Roles

In `AuthDbContext.OnModelCreating`, three roles are seeded:

- Admin (Id: "1", NormalizedName: "ADMIN")
- User (Id: "2", NormalizedName: "USER")
- Agent (Id: "3", NormalizedName: "AGENT")

5.1.5 Authentication Configuration

Identity Setup

The service adds Identity services with:

```
1 builder.Services.AddIdentity<ApplicationUser, IdentityRole>()  
2     .AddEntityFrameworkStores<AuthDbContext>()  
3     .AddDefaultTokenProviders();
```

JWT Authentication

JWT Bearer authentication is configured with the scheme name "Bearer":

- **ValidateIssuer:** Enabled, using `Jwt:Issuer`
- **ValidateAudience:** Enabled, using `Jwt:Audience`
- **ValidateLifetime:** Enabled
- **ValidateIssuerSigningKey:** Enabled
- **IssuerSigningKey:** A `SymmetricSecurityKey` created from `Jwt:Key` using UTF8 encoding

Google OAuth

Google external authentication is added with:

- `ClientId`: From `Google:ClientId` configuration
- `ClientSecret`: From `Google:ClientSecret` configuration
- `CallbackPath`: Set to `"/api/auth/google-callback"`

5.1.6 API Endpoints

AuthController Endpoints

The `AuthController` is routed at `api/[controller]` which resolves to `/api/auth`. All endpoints are unauthenticated unless otherwise specified.

POST `/api/auth/register` Registers a new user. Accepts a `RegisterRequest` with the following fields:

- `Email` (required, valid email format)
- `Password` (required, minimum length 6 characters)
- `FullName` (optional)

The endpoint checks for an existing user with the same email. If none exists, it creates an `ApplicationUser` with `UserName` set to the email, `EmailConfirmed` set to true, and the provided `FullName`. It then creates the user with the provided password. On success, returns 200 OK with `{message, userId}`. On failure, returns 400 Bad Request with error details.

POST `/api/auth/login` Authenticates an existing user. Accepts a `LoginRequest` with `Email` and `Password`. Finds the user by email, verifies the password, and returns a JWT token valid for one hour. On failure, returns 401 Unauthorized with an error message.

GET `/api/auth/google-login` Initiates the Google OAuth flow. Returns a challenge to the Google external authentication provider. The redirect URL is set to `/api/auth/google-callback` via `Url.Action("GoogleResponse", "Auth")`.

GET `/api/auth/google-callback` Handles the Google OAuth callback. Retrieves external login information via `GetExternalLoginInfoAsync()`. If the user already has an external login, they are signed in. If no user exists, the endpoint extracts the email from the Google principal, creates a new `ApplicationUser` with `EmailConfirmed = true`, links the external login via `AddLoginAsync`, and signs the user in. Returns a JWT token.

POST `/api/auth/refresh` Refreshes a JWT token. Reads the user's email from the current JWT claims and issues a new token. If no email claim is present, returns 401 Unauthorized. Returns 200 OK with a new JWT token.

GET /api/auth/me Returns information about the currently authenticated user. Calls `GetUserAsync(User)` and returns `id`, `email`, `userName`, and `fullName`. If no authenticated user is found, returns 401 Unauthorized.

POST /api/auth/logout Signs the user out. Calls `SignInManager.SignOutAsync()` and returns 200 OK with a success message.

AgentsController Endpoints

The `AgentsController` is routed at `api/[controller]` which resolves to `/api/agents`. These endpoints manage service accounts (agents such as MCP servers or LLM services).

POST /api/agents/register Requires `[Authorize(Roles = "Admin")]`. Registers a new agent service account. Accepts an `AgentRegistrationRequest` with `Name`, `Email`, and `Password`. Creates an `ApplicationUser` with the provided name as `UserName` and the provided email. After creating the user, adds the "Agent" role. On success, returns 200 OK with `{message, agentId}`.

POST /api/agents/authenticate Authenticates an agent service account. Accepts an `AgentAuthRequest` with `Name` and `Password`. Finds the user by `UserName`, verifies the password, and returns a JWT token. The token includes a `NameIdentifier` claim containing the user's ID. On failure, returns 401 Unauthorized.

5.1.7 JWT Generation

The service has two JWT generation methods:

`AuthController.GenerateJwt(string email)`

Creates a token with:

- `issuer`: From `Jwt:Issuer` configuration
- `audience`: From `Jwt:Audience` configuration
- `claims`: A single claim of type `ClaimTypes.Email` with the provided email
- `expires`: `DateTime.UtcNow.AddHours(1)`
- `signingCredentials`: HMAC-SHA256 with the key from `Jwt:Key`

`AgentsController.GenerateJwt(ApplicationUser user)`

Creates a token with:

- `issuer`: From `Jwt:Issuer` configuration
- `audience`: From `Jwt:Audience` configuration
- `claims`: A single claim of type `ClaimTypes.NameIdentifier` with the user's ID

- expires: `DateTime.UtcNow.AddHours(1)`
- signingCredentials: HMAC-SHA256 with the key from `Jwt:Key`

5.1.8 Rate Limiting

The service includes rate limiting configured in `Program.cs`:

- Policy name: `"fixed"`
- Window: `1 minute`
- Permit limit: `100 requests per window`

The rate limiter is applied via `app.UseRateLimiter()` in the middleware pipeline.

5.1.9 Database Migration Strategy

At startup, the service attempts to apply pending migrations with `dbContext.Database.Migrate()`. If this fails, it logs the error and falls back to `dbContext.Database.EnsureCreated()` which creates the database schema if no tables exist. This fallback is convenient for development but carries the warning that it cannot apply incremental migrations.

5.1.10 Logging

The service includes commented-out Serilog configuration in `Program.cs`. The `appsettings.json` file contains a Serilog section with minimum level configuration and a Console output sink. The actual Serilog setup is currently commented and not active.

5.1.11 External Dependencies

The Auth service depends on the following NuGet packages (from `Auth.csproj`):

- `Microsoft.AspNetCore.Identity.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.Sqlite`
- `Npgsql.EntityFrameworkCore.PostgreSQL`
- `Microsoft.AspNetCore.Authentication.JwtBearer`
- `Microsoft.AspNetCore.Authentication.Google`
- `DotNetEnv`
- Serilog (commented)

5.1.12 Summary

The Auth service as implemented provides:

- User registration and login with email and password
- JWT token issuance with one-hour expiration
- Google OAuth integration for external authentication
- Agent (service account) management restricted to Admin role
- ASP.NET Core Identity with PostgreSQL (production) or SQLite (development)
- Rate limiting of 100 requests per minute
- Automatic database migration at startup with fallback to schema creation

The service uses a symmetric signing key for JWT tokens and does not implement refresh token persistence (tokens are stateless). The Orchestrator's authentication dependencies (`get_user_context` and `require_admin` referenced in `websocket_router.py` and `session_router.py`) expect to validate JWTs issued by this Auth service.